My Food Business

Master Plan — Version 4.0

Website First → Android App → WhatsApp → Game Module → Full Launch

Mobile Website	Android App	WhatsApp
Orders, tracking, games — works on every phone	For loyal repeat customers — richer experience	Location sharing, order updates, new customers

Total timeline: 14 weeks | Version 2026

1. Strategy — What You're Building & Why

1.1 The Three Channels

Channel	Primary Users	Why It Matters
Mobile Website (React)	New customers discovering you	No download needed. Google finds you. WhatsApp link opens instantly. Fastest to launch.
Android App (React Native)	Loyal repeat customers	App icon on home screen. Push notifications. Richer game experience. Build after website is live.
WhatsApp Bot	Anyone who messages you directly	Huge in India. Location sharing built-in. Same backend, different front-door.

1.2 Build Order — Website First, App Second

☑ This is the correct sequence. Website gives you revenue immediately. App is built while business is running.

Phase	What Ships	Who Can Order
Week 6 →	Mobile website live	100% of customers (any phone, any OS)
Week 8 →	WhatsApp ordering live	WhatsApp users — biggest channel in India
Week 11 →	Android app live	Android users who want native app experience
Week 14 →	Game module in both website + app	All customers — coupon rewards drive repeat orders

1.3 iOS — Eliminated for Now

ℹ iOS requires a Mac for builds and a $99/year Apple Developer account. Android covers 75–80% of India. Add iOS after launch if demand exists — the React Native code already supports it, only the build + store submission process is extra.

2. Final Tech Stack

Layer	Technology	Used In	Why
Customer Website	React + Vite + Tailwind CSS	Website	Mobile-first, fast, SEO-friendly, Tailwind makes it look great
Customer App	React Native (Expo)	Android App	Same React knowledge. One codebase. $25 Play Store fee only.
Admin Dashboard	React + Vite + Shadcn/ui	Web only	Manage orders, menu, agents, game config from browser
Backend API	Spring Boot (Java)	All channels	Your existing skill. REST + WebSocket. Handles everything.
Database	PostgreSQL on Railway	All channels	Free tier. Managed. No MongoDB needed.
Auth	JWT + OTP + BCrypt(12) + Salt	All channels	Secure. Guest checkout supported. No mandatory login.
Payments	Razorpay	Website + App	2% per transaction. UPI + card + COD. Best for India.
Real-time	Spring WebSocket	Tracking	Live order status + delivery GPS. Built into Spring Boot.
Push Alerts	Firebase FCM	App + Website	Free. Android push + browser push for website.
OTP SMS	MSG91	All channels	~₹0.18/SMS. Indian provider. Reliable delivery.
WhatsApp	AiSensy	WhatsApp channel	No-code bot builder. Webhooks to Spring Boot. Free 1k/month.
Maps	Google Maps Platform	Website + App	$200 free credit/month. Live tracking + address picker.
Images	Cloudflare R2	Website + App	10GB free. Menu photos, food images.
Hosting (API)	Railway	Backend	Git push deploy. Auto SSL. Free tier generous for early stage.
Hosting (Web + Admin)	Vercel	Website + Admin	Zero config. Free forever for basic use.
CI/CD	GitHub Actions	All	Auto deploy on push to main.
Crash reporting	Sentry	All	Free tier. Catches errors before customers complain.

3. Accounts to Create (Before You Start Coding)

Create all of these before Week 1. Most take 5–15 minutes. Some (Razorpay, Google Cloud) take 1–2 days for verification.

Service	URL	When to Create	Verification Needed?
GitHub	github.com	Day 1	No — instant
Railway	railway.app	Day 1	No — instant
Vercel	vercel.com	Day 1	No — instant
Firebase	console.firebase.google.com	Day 1	No — Google account
Cloudflare	cloudflare.com	Day 1	No — instant
Razorpay	razorpay.com	Day 1–2	Yes — business PAN + bank account (2–3 days)
Google Cloud Console	console.cloud.google.com	Day 1	Credit card for billing (not charged under $200/month)
MSG91	msg91.com	Week 1	Yes — DLT registration (~2–3 days for SMS in India)
AiSensy	aisensy.com	Week 7	Yes — WhatsApp Business verification (1–3 days)
Google Play Console	play.google.com/console	Week 10	$25 one-time fee
Expo / EAS	expo.dev	Week 9	No — free account
Sentry	sentry.io	Week 11	No — instant
Domain (e.g. GoDaddy)	godaddy.com / namecheap.com	Week 5	₹800–1,200/year

⚠ Start Razorpay and MSG91 registration on Day 1 — both require business verification that takes 2–3 days. You cannot test payments or OTPs until these are approved.

4. Cost — Development Phase to Post-Launch

4.1 During Development (Weeks 1–14): Near Zero

Service	Free Tier	Cost During Dev
Railway (backend + DB)	$5 credit/month	₹0 — free credit covers dev usage
Vercel (website + admin)	Unlimited hobby	₹0 forever
Firebase FCM	Unlimited	₹0 forever
Google Maps API	$200/month credit	₹0 — dev traffic is minimal
Cloudflare R2	10GB + 1M requests	₹0 during dev
Razorpay	Free to set up	₹0 — only pay when orders come in
MSG91 OTP	Pay per SMS	~₹200 for testing (~1,000 OTPs)
AiSensy	1,000 conversations/month	₹0 during dev
Expo EAS Build	15 builds/month	₹0 during dev
GitHub	Free for public/private repos	₹0
Sentry	5,000 errors/month free	₹0
Domain name	No free tier	₹800–1,200 one-time
Google Play Console	One-time fee	₹2,100 one-time (~$25)

Total out-of-pocket before first order arrives	**~₹3,300–3,500**

4.2 After Launch — Monthly Running Cost

Service	Free Until	Paid Tier Cost
Railway	~500 daily orders fine on free	~$5–10/month if traffic grows significantly
Vercel	Free forever for this use case	₹0
Google Maps	~900 delivery pings/day free	$0.005 per extra map load after limit
MSG91 OTP	Pay per use	~₹0.18 per OTP — ₹500 lasts 2,700 OTPs
AiSensy WhatsApp	1,000 conversations free	₹999/month for up to 10,000 conversations
Razorpay	No monthly fee	2% per order — you pay only when earning
Firebase FCM	Unlimited forever	₹0
Cloudflare R2	10GB free	₹1.3/GB after 10GB — never hit early on

☑ Realistically your first 3–6 months of operation cost under ₹1,000/month in infrastructure. The only real ongoing cost is Razorpay 2% per order — which means you only pay when revenue is coming in.

5. Week-by-Week Development Timeline

This is the exact plan — what you build every week, in what order, and what is usable at the end of each week.

Phase	Color	Weeks	Focus
Foundation	Blue	1–2	Backend + DB + auth + guest checkout
Website	Green	3–6	Mobile website — full customer ordering flow
Operations	Orange	7–8	Admin dashboard + delivery agent app + WhatsApp
Android App	Purple	9–11	React Native app — mirrors website features
Game Module	Red	12–13	Penalty Shootout in website + app + admin config
Launch	Black	14	Polish, testing, Play Store, go live

Week	Phase	What You Build	End of Week
Week 1	**Foundation**	Create all accounts (Railway, Vercel, Firebase, Razorpay, MSG91, Google Cloud). Spring Boot project setup. PostgreSQL schema — all tables (users, orders, order_items, menu, coupons, game tables, tracking, whatsapp_orders). BCrypt + three-layer salt password service. JWT auth. OTP send + verify endpoints.	**All accounts ready. DB schema live on Railway. Auth APIs tested in Postman.**
Week 2	**Foundation**	Guest checkout API (/checkout/guest). Order placement with guest_token. tracking_token generation. Razorpay order create + verify APIs. Order status transitions (PLACED→CONFIRMED→PREPARING→READY→PICKED_UP→DELIVERED). Public tracking endpoint (/track/{token}). Deploy backend to Railway. GitHub Actions CI/CD.	**Guest can place and pay for an order via Postman. Tracking URL returns live status.**
Week 3	**Website**	React + Vite + Tailwind project setup on Vercel. Mobile-first layout (header, nav, footer). Home page — your branding, hero, featured items. Menu page — categories sidebar + item grid. Single item detail page. Cart (React context/state — items, quantities, total). Cart drawer/sheet component.	**Website loads on phone. Menu browsable. Items add to cart. Deployed on Vercel.**
Week 4	**Website**	Checkout page — guest form (name, phone, address). Google Maps Places Autocomplete for address. GPS "Use my location" button. OTP verification step (phone → OTP → verified). Payment page — Razorpay Web SDK (UPI + card +	**Full guest checkout flow works end-to-end.**

Week	Phase	What You Build	End of Week
		COD). Order confirmation page with order number + tracking link. "Save my details" checkbox.	**Real Razorpay test payment processes.**
Week 5	**Website**	Live order tracking page (/track/{token}) — status timeline + Google Maps pin for agent location. WebSocket connection to backend for real-time status. SMS notification on order placed + status changes (MSG91). Optional login/signup page. Returning user: saved addresses + order history + one-tap reorder. Domain setup + SSL.	**Guest gets tracking link via SMS. Live map pin updates as agent moves.**
Week 6	**Website**	Mobile UX polish pass — bottom nav on mobile, tap targets, loading skeletons, empty states. Error handling on all forms. Offline message if connection drops. SEO — meta tags, OpenGraph for WhatsApp link previews, sitemap.xml. Performance — image lazy loading, Tailwind purge. Cross-browser test on Android Chrome + Samsung Browser.	🚀 **WEBSITE LIVE — customers can order from phone browser. Share link on WhatsApp.**
Week 7	**Operations**	Admin dashboard (React + Vite + Shadcn). Login screen (admin only). Dashboard home — orders today, revenue today, pending orders count. Orders page — full list with status filter + date range. Order detail view — items, customer, address, assign agent button. Menu management — categories + items CRUD + image upload to Cloudflare R2.	**You can see and manage all orders from your laptop. Menu editable without code.**
Week 8	**Operations**	Delivery agent mobile screen (Expo — quick separate app). Agent login. Incoming order notification. Accept/reject. Start delivery → GPS ping every 30s to delivery_tracking table. Mark delivered. WhatsApp Business API setup via AiSensy — greeting bot, location request, menu reply, order confirmation, tracking link sent automatically.	🚀 **WHATSAPP ORDERING LIVE. Agent receives orders on phone. Customer gets tracking link on WhatsApp.**
Week 9	**Android App**	Expo React Native project setup. Reuse all existing API calls from website (same backend). Bottom tab navigation (Home, Orders, Profile, Games). Home screen + menu screen — 80% same logic as website, adapted for native. Cart screen. Shared custom hooks for API calls between website and app.	**Android app builds and runs on physical device. Menu browsable natively.**

Week	Phase	What You Build	End of Week
Week 10	**Android App**	Checkout flow in app — OTP login, saved addresses, Razorpay React Native SDK. Order tracking screen with Google Maps native — smoother than website version. Firebase FCM push notifications (order updates, game rewards). Order history + reorder. Profile screen — saved addresses, account settings.	**Full ordering flow works in Android app. Push notifications firing.**
Week 11	**Android App**	App UI polish — animations (React Native Reanimated), haptic feedback, splash screen, app icon. EAS Build setup — generate signed APK for Android. Internal testing via Google Play Console internal track (share with 5–10 testers). Fix bugs from real device testing. Admin dashboard: customer list, coupon management, revenue chart (Recharts).	**🚀 ANDROID APP in internal testing. Admin has full analytics view.**
Week 12	**Game Module**	All game DB tables already exist from Week 1. Game APIs — OTP entry, session token, penalty kick/dive/finish, HMAC win token, coupon generation. Level progression service. Cooldown enforcement. Penalty Shootout game UI in WEBSITE (React) — animated goalkeeper + ball using CSS animations. Phone OTP entry screen. Level badge display.	**Game playable on website. Coupons generated and redeemable at checkout.**
Week 13	**Game Module**	Penalty Shootout UI in ANDROID APP (React Native Reanimated — smoother animations than web). Active coupon countdown timer in both website + app. Admin game config UI — edit discount value + expiry hours per level (PUT /admin/game/levels/{level}). Admin game analytics — plays/day, win rate, coupon redemption rate. Coupon apply at checkout fully validated server-side.	**🎮 GAME LIVE on both website and app. Admin controls rewards without code changes.**
Week 14	**Launch**	Security audit — OTP brute force test, win token replay test, rate limit verification. Performance — Lighthouse score on website (target 90+), API response time under 200ms for all key endpoints. Sentry integration for crash reporting. UptimeRobot for API uptime monitoring. Play Store full listing (screenshots, description, privacy policy). Submit for review.	**🚀 FULL PUBLIC LAUNCH — Website + Android App + WhatsApp + Game all live.**

6. Key Milestones

🌐 **MILESTONE 1 — End of Week 6: Mobile Website Live → First real customer order possible**

💬 **MILESTONE 2 — End of Week 8: WhatsApp Ordering Live → Customers order via WhatsApp message**

📱 **MILESTONE 3 — End of Week 11: Android App in Testing → Internal testers using native app**

🎮 **MILESTONE 4 — End of Week 13: Game Module Live → Customers win coupons, repeat orders increase**

🚀 **MILESTONE 5 — End of Week 14: Full Public Launch → Website + App + WhatsApp + Game all live**

7. What Each Channel Does — Final Summary

Feature	Website	Android App	WhatsApp
Browse menu	☑	☑	☑ Text-based
Guest checkout (no login)	☑	☑	☑
OTP login (optional)	☑	☑	Phone verified naturally
Address via GPS	☑	☑	Location share button
Address via Google Search	☑	☑	✗
Razorpay payment	☑	☑	UPI link sent by bot
Live order tracking map	☑	☑ (smoother)	Tracking link in message
Push notifications	☑ (browser)	☑ (FCM)	WhatsApp messages
Order history	☑ (if logged in)	☑	✗
Play game + win coupon	☑	☑ (richer UI)	✗
Google can find you	☑ SEO	✗	✗
No download required	☑	✗	☑
Works on iOS	☑	✗ (Phase 2)	☑

8. Game Module — Penalty Shootout

8.1 Quick Summary

Attribute	Value
Game	Penalty Shootout (3 kicks each vs AI)
Win condition	Score more goals than AI across both halves
Session length	30–90 seconds
Entry requirement	Phone number + OTP (no account needed)
Reward	Discount coupon — amount set by admin per level
Coupon format	FOOD-L3-PS-9AK4 (level + game type + random suffix)
Cooldown	Level-dependent — 6–24 hours between plays per phone
Cheating prevention	AI direction server-side only. Win state HMAC-signed. Token single-use.
Admin control	Discount value + expiry hours per level editable from dashboard

8.2 Level Rewards (Admin Configurable)

Level	Name	Unlocks After	Default Reward	Coupon Expires	Cooldown
1	Newbie	0 games	5% off	24 hours	24 hours
2	Explorer	5 games	10% off	36 hours	12 hours
3	Regular	15 games	15% off	48 hours	12 hours
4	Foodie	30 games	₹50 flat off	72 hours	8 hours
5	Champion	50 games	₹100 flat off	96 hours	6 hours

ℹ Admin changes discount values from the dashboard. No code change or redeployment needed. Changes apply to the next coupon generated for that level.

8.3 Security Rules (Non-Negotiable)

🔒 All of these are enforced server-side. Client UI is cosmetic only for all security rules.

Rule	Implementation
AI goalkeeper direction never sent to client before kick	Computed server-side AFTER player commits direction. Sent only in same response as goal result.

Rule	Implementation
Win state cannot be forged	win_token = HMAC-SHA256(session_id + phone + timestamp, server_secret). Client cannot generate valid token.
Win token is single-use	win_token_used flag set on first coupon redemption. Replay attempts return 400.
Cooldown cannot be bypassed	cooldown_until stored in DB. Checked server-side on every game start request. Client timer is UI only.
One active coupon maximum	Server blocks new game if active_coupon_id is not null and coupon not expired.
OTP rate limited	Max 3 OTP requests per phone per hour + max 10 per IP per hour.
Game session expires	game_session_token is JWT with 10-minute TTL. Expired sessions rejected.

9. Guest Checkout & WhatsApp Location

9.1 Guest Checkout Flow

Step	What Happens
1 — Cart	Customer taps "Checkout" — no login prompt shown
2 — Details	Form: Name (required), Phone (required), Email (optional), Address (GPS or search)
3 — Payment	Razorpay opens — UPI / card / COD
4 — Confirmed	Order number + tracking link shown on screen
5 — SMS/WhatsApp	Tracking link sent to phone automatically
6 — Save offer	"Save my details for next time?" checkbox — tick = silent account creation

ℹ Phone number collected even for guests — used for order updates, OTP game entry, and WhatsApp notifications. It is a contact field, not a login requirement.

9.2 WhatsApp Location Sharing

The WhatsApp channel works as a fully separate order entry point. Customer messages your WhatsApp Business number → bot guides them through ordering → they share their location → delivery agent gets GPS coordinates.

Step	Customer Does	Your System Does
1	Messages your WhatsApp: "Hi I want to order"	AiSensy bot greets them, asks for location
2	Taps Attachment → Location → Share current location	Webhook fires to Spring Boot with lat/lng coordinates
3	Receives menu via WhatsApp message	Bot sends menu summary or PDF
4	Replies with order: "1 Biryani 2 Roti"	Bot parses order, creates record in whatsapp_orders table
5	Receives total + UPI payment link	Backend generates Razorpay payment link, sends via WhatsApp
6	Pays via UPI link	Payment webhook confirms → order created in orders table
7	Receives order confirmation + tracking link	Tracking link works same as website — no app needed
8	Delivery agent opens admin — sees GPS pin	Coordinates from step 2 shown on agent's map

☑ The location coordinates from WhatsApp are stored in the order. The delivery agent sees a Google Maps link directly in the admin panel. No extra engineering overhead — AiSensy handles all the WhatsApp conversation logic.

— End of Document —